

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (Tiếng Việt)

Aurélien Géron — bản dịch chỉnh sửa (2nd ed.)

Ngày 6 tháng 7 năm 2026

Mục lục

0.1	Những thách thức chính của Machine Learning	3
0.2	Không đủ số lượng dữ liệu training	3
0.3	Hiệu quả phi lý của dữ liệu	4
0.4	Mục lục	4
0.5	Ví dụ về lấy mẫu bias	4
0.6	Dữ liệu kém chất lượng	5
0.7	Tính năng không liên quan	5
0.8	Trang bị quá mức dữ liệu training	6
0.9	Trang bị thiếu dữ liệu training	7
0.10	Bước lùi	7
0.11	Điều chỉnh siêu tham số và lựa chọn model	8
0.12	Dữ liệu không khớp	9
0.13	Định lý không có bữa trưa miễn phí	9
1	Bài tập	10
1.1	Chương 2: Dự án Machine Learning từ đầu đến cuối	10
1.2	Làm việc với dữ liệu thực	11
1.3	Nhìn vào bức tranh lớn	11
1.4	Đóng khung vấn đề	11
1.5	Đường ống	12
1.6	Chọn thước đo hiệu suất	13
1.7	Ký hiệu	13
1.8	Kiểm tra các giả định	14
1.9	Lấy dữ liệu	14
1.10	Tạo không gian làm việc	14
1.11	Tạo một môi trường biệt lập	15
1.12	Tải xuống dữ liệu	15
1.13	Xem nhanh cấu trúc dữ liệu	16
1.14	Tạo một bộ thử nghiệm	17
1.15	Trực quan hóa dữ liệu địa lý	18
1.16	Tìm kiếm mối tương quan	19
1.17	Thử nghiệm với sự kết hợp thuộc tính	20
1.18	Chuẩn bị dữ liệu cho thuật toán Machine Learning	20

1.19	Làm sạch dữ liệu	20
1.20	Thiết kế Scikit-Learn	21
1.21	Xử lý thuộc tính văn bản và classification	21
1.22	Custom Transformer	22
1.23	Chia tỷ lệ tính năng	23
1.24	Đường ống chuyển đổi	23

Bây giờ model khớp với dữ liệu training nhất có thể (đối với model tuyến tính).

Cuối cùng bạn đã sẵn sàng chạy model để đưa ra dự đoán. Ví dụ, giả sử bạn muốn biết người dân Síp hạnh phúc đến mức nào, và dữ liệu của OECD không có câu trả lời. May mắn thay, bạn có thể sử dụng model của mình để đưa ra một dự đoán tốt: bạn tra cứu GDP bình quân đầu người của Síp, thấy con số là 22.587 đô la, sau đó áp dụng model của mình và thấy rằng mức độ hài lòng về cuộc sống có thể nằm trong khoảng $4,85 + 22.587 \times 4,91 \times 10^{-5} = 5,96$.

Ví dụ 1-1 hiển thị code Python load dữ liệu, chuẩn bị dữ liệu, tạo một biểu đồ phân tán để trực quan hóa, sau đó training model tuyến tính và đưa ra dự đoán.

Nếu bạn sử dụng thuật toán instance-based learning, bạn sẽ thấy rằng Slovenia có GDP bình quân đầu người gần nhất với Cyprus (20.732 đô la), và vì dữ liệu của OECD cho thấy mức độ hài lòng về cuộc sống của người Slovenia là 5,7, bạn sẽ dự đoán mức độ hài lòng về cuộc sống của Cyprus là 5,7. Nếu bạn nhìn rộng hơn một chút và xem xét hai quốc gia gần nhất tiếp theo, bạn sẽ thấy Bồ Đào Nha và Tây Ban Nha có mức độ hài lòng về cuộc sống lần lượt là 5,1 và 6,5. Trung bình cộng ba giá trị này, bạn sẽ nhận được 5,77, khá gần với dự đoán dựa trên model của bạn. Thuật toán đơn giản này được gọi là regression k-Nearest Neighbors (trong ví dụ này, $k = 3$).

Việc thay thế Linear Regression model bằng k-Nearest Neighbors regression trong code trước cũng đơn giản như thay thế hai dòng sau:

Nếu mọi việc suôn sẻ, model của bạn sẽ đưa ra những dự đoán tốt. Nếu không, bạn có thể cần sử dụng nhiều thuộc tính hơn (tỷ lệ việc làm, sức khỏe, ô nhiễm không khí, v.v.), nhận được nhiều dữ liệu training có chất lượng tốt hơn hoặc có thể chọn model mạnh hơn (ví dụ: Polynomial Regression model).

Tóm lại:

- Bạn đã nghiên cứu dữ liệu.

Ấn bản thứ hai

- Bạn đã training nó trên dữ liệu training (tức là thuật toán học đã tìm kiếm các giá trị model parameter để cực tiểu hóa hàm chi phí).

- Cuối cùng, bạn áp dụng model để đưa ra dự đoán cho các trường hợp mới (gọi là suy luận), hy vọng model này sẽ khá tốt.

Đây là giao diện của một dự án Machine Learning điển hình. Trong Chương 2, bạn sẽ trực tiếp trải nghiệm điều này bằng cách xem xét từ đầu đến cuối dự án.

Cho đến nay, chúng ta đã đề cập đến rất nhiều kiến thức: bây giờ bạn đã biết Machine Learning thực sự là gì, tại sao nó hữu ích, một số danh mục phổ biến nhất của hệ thống ML là gì và quy trình làm việc dự án điển hình trông như thế nào. Bây giờ, hãy xem điều gì có thể xảy ra sai sót trong quá trình học tập và ngăn cản bạn đưa ra những dự đoán chính xác.

0.1 Những thách thức chính của Machine Learning

Nói tóm lại, vì nhiệm vụ chính của bạn là chọn một thuật toán học và training nó trên một số dữ liệu, nên hai điều có thể sai là “thuật toán xấu” và “dữ liệu xấu”. Hãy bắt đầu với các ví dụ về dữ liệu xấu.

0.2 Không đủ số lượng dữ liệu training

Để trẻ mới biết đi biết quả táo là gì, bạn chỉ cần chỉ vào quả táo và nói “quả táo” (có thể lặp lại quy trình này một vài lần). Bây giờ trẻ đã có thể nhận biết được các loại táo với

đủ loại màu sắc và hình dạng. Thiên tài.

Machine Learning vẫn chưa hoàn thiện; cần rất nhiều dữ liệu để hầu hết các thuật toán Machine Learning hoạt động bình thường. Ngay cả đối với những vấn đề rất đơn giản, bạn thường cần hàng nghìn ví dụ và đối với những vấn đề phức tạp như nhận dạng hình ảnh hoặc giọng nói, bạn có thể cần hàng triệu ví dụ (trừ khi bạn có thể sử dụng lại các phần của model hiện có).

0.3 Hiệu quả phi lý của dữ liệu

Trong một bài báo nổi tiếng xuất bản năm 2001, các nhà nghiên cứu Michele Banko và Eric Brill của Microsoft đã chỉ ra rằng các thuật toán Machine Learning rất khác nhau, bao gồm cả những thuật toán khá đơn giản, hoạt động gần như giống hệt nhau đối với một vấn đề phức tạp về phân định ngôn ngữ tự nhiên khi chúng được cung cấp đủ dữ liệu.

Như các tác giả đã nói, “những kết quả này cho thấy rằng chúng ta có thể muốn xem xét lại sự cân bằng giữa việc dành thời gian và tiền bạc cho việc phát triển thuật toán so với việc dành nó cho việc phát triển kho ngữ liệu”.

Indexer: Judith McConville | Interior designer: David Futato | Cover designer: Karen Montgomery | Illustrator: Rebecca Demarest

0.4 Mục lục

Để khái quát hóa tốt, điều quan trọng là dữ liệu training của bạn phải đại diện cho các trường hợp mới mà bạn muốn khái quát hóa. Điều này đúng cho dù bạn sử dụng phương pháp instance-based learning hay model-based learning.

Ví dụ: tập hợp các quốc gia mà chúng ta sử dụng trước đó để training model tuyến tính không mang tính đại diện hoàn hảo; một vài quốc gia đã bị mất tích.

Nếu bạn training model tuyến tính trên dữ liệu này, bạn sẽ có được đường liền nét, trong khi model cũ được biểu thị bằng đường chấm. Như bạn có thể thấy, việc thêm một vài quốc gia bị thiếu không chỉ làm thay đổi đáng kể model mà còn cho thấy rõ ràng model tuyến tính đơn giản như vậy có lẽ sẽ không bao giờ hoạt động tốt. Có vẻ như các nước rất giàu không hạnh phúc hơn các nước giàu vừa phải (trên thực tế, họ có vẻ bất hạnh hơn), và ngược lại một số nước nghèo có vẻ hạnh phúc hơn nhiều nước giàu.

Bằng cách sử dụng training set không đại diện, chúng ta đã training model không có khả năng đưa ra dự đoán chính xác, đặc biệt là đối với các nước rất nghèo và rất giàu.

Điều quan trọng là sử dụng training set đại diện cho các trường hợp bạn muốn khái quát hóa. Điều này thường khó hơn tưởng tượng: nếu mẫu quá nhỏ, bạn sẽ có nhiều lấy mẫu (tức là dữ liệu không mang tính đại diện do ngẫu nhiên), nhưng ngay cả những mẫu rất lớn cũng có thể không mang tính đại diện nếu phương pháp lấy mẫu bị sai sót. Điều này được gọi là lấy mẫu bias.

0.5 Ví dụ về lấy mẫu bias

Có lẽ ví dụ nổi tiếng nhất về việc lấy mẫu bias đã xảy ra trong cuộc bầu cử tổng thống US năm 1936, khiến Landon đọ sức với Roosevelt: Literary Digest đã tiến hành một cuộc thăm dò rất lớn, gửi thư tới khoảng 10 triệu người. Nó đã nhận được 2,4 triệu câu trả lời và dự đoán với độ tin cậy cao rằng Landon sẽ nhận được 57% số phiếu bầu. Thay vào đó,

Roosevelt đã giành chiến thắng với 62% số phiếu bầu. Lỗi hổng nằm ở phương pháp lấy mẫu của Literary Digest:

- Đầu tiên, để có được địa chỉ gửi các cuộc thăm dò ý kiến, Literary Digest đã sử dụng danh bạ điện thoại, danh sách người đăng ký tạp chí, danh sách thành viên câu lạc bộ, v.v. Tất cả những danh sách này đều có xu hướng ủng hộ những người giàu có hơn, những người có nhiều khả năng bỏ phiếu cho đảng Cộng hòa hơn (do đó là Landon).

- Thứ hai, chưa đến 25% số người được hỏi trả lời. Một lần nữa, điều này đã giới thiệu một mẫu bias, bằng cách có khả năng loại trừ những người không quan tâm nhiều đến chính trị, những người không thích Literary Digest và các nhóm chủ chốt khác. Đây là kiểu lấy mẫu bias đặc biệt được gọi là bias không phản hồi.

Đây là một ví dụ khác: giả sử bạn muốn xây dựng một hệ thống nhận dạng các video nhạc funk. Một cách để xây dựng training set của bạn là tìm kiếm “nhạc funk” trên YouTube và sử dụng các video thu được. Nhưng điều này giả định rằng công cụ tìm kiếm của YouTube trả về một tập hợp các video đại diện cho tất cả các video nhạc funk trên YouTube. Trên thực tế, kết quả tìm kiếm có thể thiên về các nghệ sĩ nổi tiếng (và nếu bạn sống ở Brazil, bạn sẽ nhận được rất nhiều video “funk carioca”, nghe chẳng giống James Brown chút nào). Mặt khác, làm cách nào khác bạn có thể có được một chiếc training set lớn?

0.6 Dữ liệu kém chất lượng

Rõ ràng, nếu dữ liệu training của bạn chứa đầy lỗi, ngoại lệ và nhiễu (ví dụ: do các phép đo chất lượng kém), điều đó sẽ khiến hệ thống khó phát hiện các mẫu cơ bản hơn, do đó hệ thống của bạn ít có khả năng hoạt động tốt. Việc dành thời gian dọn dẹp dữ liệu training của bạn thường rất đáng giá. Sự thật là hầu hết các nhà khoa học dữ liệu đều dành một phần đáng kể thời gian của họ để làm việc đó. Sau đây là một số ví dụ về thời điểm bạn muốn dọn sạch dữ liệu training:

- Nếu một số trường hợp rõ ràng là ngoại lệ, có thể chỉ cần loại bỏ chúng hoặc cố gắng sửa lỗi theo cách thủ công.

- Nếu một số phiên bản thiếu một vài features (ví dụ: 5% khách hàng của bạn không chỉ định tuổi của họ), bạn phải quyết định xem bạn có muốn bỏ qua thuộc tính này hoàn toàn hay không, bỏ qua các phiên bản này, điền vào các giá trị còn thiếu (ví dụ: với độ tuổi trung bình) hoặc training một model với feature và một model không có thuộc tính đó.

0.7 Tính năng không liên quan

Tục ngữ có câu: rác vào, rác ra. Hệ thống của bạn sẽ chỉ có khả năng học nếu dữ liệu training chứa đủ features phù hợp và không có quá nhiều dữ liệu không liên quan. Một phần quan trọng tạo nên sự thành công của dự án Machine Learning là có được một bộ features tốt để training. Quá trình này, được gọi là kỹ thuật feature, bao gồm các bước sau:

- Lựa chọn Feature (chọn features hữu ích nhất để training trong số features hiện có)
- Trích xuất Feature (kết hợp features hiện có để tạo ra một features hữu ích hơn—như chúng ta đã thấy trước đó, các thuật toán giảm kích thước có thể giúp ích)
- Tạo features mới bằng cách thu thập dữ liệu mới

Bây giờ chúng ta đã xem xét nhiều ví dụ về dữ liệu xấu, hãy xem một vài ví dụ về thuật toán xấu.

0.8 Trang bị quá mức dữ liệu training

Giả sử bạn đang đi du lịch nước ngoài và tài xế taxi đã lừa bạn. Bạn có thể muốn nói rằng tất cả tài xế taxi ở nước đó đều là kẻ trộm. Khái quát hóa quá mức là điều mà con người chúng ta thường xuyên làm, và thật không may, máy móc có thể rơi vào bẫy tương tự nếu chúng ta không cẩn thận. Trong Machine Learning, nó được gọi là overfitting: có nghĩa là model hoạt động tốt trên dữ liệu training nhưng không khái quát hóa tốt.

Các model phức tạp như neural network sâu có thể phát hiện các mẫu tinh tế trong dữ liệu, nhưng nếu tập training có nhiều hoặc nếu nó quá nhỏ (điều này đưa ra nhiều lấy mẫu), thì model có khả năng phát hiện các mẫu trong nhiễu. Rõ ràng các mẫu này sẽ không khái quát được cho các trường hợp mới. Ví dụ: giả sử bạn cung cấp cho model đánh giá sống hạnh phúc của bạn nhiều thuộc tính hơn, bao gồm các thuộc tính không cung cấp thông tin như tên quốc gia. Trong trường hợp đó, một model phức tạp có thể phát hiện các mẫu như việc tất cả các quốc gia trong dữ liệu training có chữ w trong tên của họ đều có đánh giá sống hạnh phúc lớn hơn 7: New Zealand (7.3), Norway (7.4), Sweden (7.2), và Switzerland (7.5). Bạn có bao nhiêu niềm tin rằng quy tắc w-satisfaction khái quát được cho Rwanda hoặc Zimbabwe? Rõ ràng mẫu này xảy ra trong dữ liệu training bởi sự may rủi, nhưng model không có cách nào để biết liệu một mẫu có phải là thực sự hay chỉ là kết quả của nhiễu trong dữ liệu.

Overfitting xảy ra khi model quá phức tạp so với số lượng và độ ồn của dữ liệu training. Dưới đây là các giải pháp khả thi:

- Đơn giản hóa model bằng cách chọn một parameters ít hơn (ví dụ: model tuyến tính thay vì model đa thức bậc cao), bằng cách giảm số lượng thuộc tính trong dữ liệu training hoặc bằng cách hạn chế model.
- Thu thập thêm dữ liệu training.
- Giảm nhiễu trong dữ liệu training (ví dụ: sửa lỗi dữ liệu và loại bỏ các giá trị ngoại lệ).

Việc ràng buộc model để đơn giản hơn và giảm rủi ro về overfitting được gọi là regularization. Ví dụ: model tuyến tính mà chúng ta đã xác định trước đó có hai parameters, θ_0 và θ_1 . Điều này mang lại cho thuật toán học hai bậc tự do để điều chỉnh model cho phù hợp với dữ liệu training: nó có thể điều chỉnh cả chiều cao (θ_0) và gradient (θ_1) của đường thẳng. Nếu chúng ta buộc $\theta_1 = 0$, thuật toán sẽ chỉ có một bậc tự do và sẽ gặp khó khăn hơn nhiều khi khớp dữ liệu một cách chính xác: tất cả những gì nó có thể làm là di chuyển dòng lên hoặc xuống để đến gần nhất có thể với các trường hợp training, vì vậy nó sẽ kết thúc quanh mức trung bình. Thực sự là một model rất đơn giản! Nếu chúng ta cho phép thuật toán sửa đổi 1 nhưng chúng ta buộc nó phải giữ nó ở mức nhỏ thì thuật toán học thực tế sẽ có khoảng từ một đến hai bậc tự do. Nó sẽ tạo ra một model đơn giản hơn một model có hai bậc tự do, nhưng phức tạp hơn một model chỉ có một bậc tự do. Bạn muốn tìm sự cân bằng phù hợp giữa việc điều chỉnh dữ liệu training một cách hoàn hảo và giữ cho model đủ đơn giản để đảm bảo rằng nó sẽ khái quát tốt.

Nhưng có ràng buộc regularization. Bạn có thể thấy rằng regularization đã buộc model phải có gradient nhỏ hơn: model này không phù hợp với dữ liệu training (hình tròn) cũng như model đầu tiên, nhưng nó thực sự khái quát hóa tốt hơn các ví dụ mới mà nó không thấy trong quá trình training (hình vuông).

Lượng regularization áp dụng trong quá trình học có thể được kiểm soát bởi siêu parameter. Hyperparameter là parameter của thuật toán learning (không phải của model). Như vậy, nó không bị ảnh hưởng bởi chính thuật toán học; nó phải được thiết lập trước

khi training và không đổi trong quá trình training. Nếu bạn đặt siêu regularization hyperparameter thành một giá trị rất lớn, bạn sẽ nhận được model gần như phẳng (gradient gần bằng 0); thuật toán học gần như chắc chắn sẽ không phù hợp quá mức với dữ liệu training nhưng sẽ ít có khả năng tìm ra giải pháp tốt hơn. Điều chỉnh hyperparameters là một phần quan trọng trong việc xây dựng hệ thống Machine Learning (bạn sẽ xem ví dụ chi tiết trong chương tiếp theo).

0.9 Trang bị thiếu dữ liệu training

Như bạn có thể đoán, underfitting trái ngược với overfitting: nó xảy ra khi model của bạn quá đơn giản để tìm hiểu cấu trúc cơ bản của dữ liệu. Ví dụ, một model tuyến tính về mức độ hài lòng với cuộc sống thường không phù hợp; thực tế chỉ phức tạp hơn model, do đó các dự đoán của nó chắc chắn sẽ không chính xác, ngay cả trên training examples.

Dưới đây là các tùy chọn chính để khắc phục sự cố này:

- Chọn model mạnh hơn, có nhiều parameters hơn.
- Cung cấp features tốt hơn cho thuật toán learning (kỹ thuật feature).
- Giảm các ràng buộc trên model (ví dụ: giảm siêu tham số regularization).

0.10 Bước lùi

Đến bây giờ bạn đã biết nhiều về Machine Learning. Tuy nhiên, chúng ta đã xem xét quá nhiều khái niệm nên có thể bạn sẽ cảm thấy hơi lạc lõng, vì vậy hãy lùi lại và nhìn vào bức tranh toàn cảnh:

- Machine Learning hướng tới việc giúp máy móc thực hiện một số nhiệm vụ tốt hơn bằng cách học hỏi từ dữ liệu, thay vì phải viết code các quy tắc một cách rõ ràng.
- Có nhiều loại hệ thống ML khác nhau: supervised hoặc không, batch hoặc trực tuyến, dựa trên phiên bản hoặc dựa trên model.
- Trong dự án ML, bạn thu thập dữ liệu trong training set và cung cấp training set cho thuật toán learning. Nếu thuật toán dựa trên model, nó sẽ điều chỉnh một số parameters để khớp model với training set (tức là để đưa ra những dự đoán tốt về chính training set), và sau đó hy vọng nó cũng có thể đưa ra những dự đoán tốt về các trường hợp mới. Nếu thuật toán dựa trên phiên bản, nó chỉ học thuộc lòng các ví dụ và khái quát hóa các phiên bản mới bằng cách sử dụng thước đo độ tương tự để so sánh chúng với các phiên bản đã học.
- Hệ thống sẽ không hoạt động tốt nếu training set của bạn quá nhỏ hoặc nếu dữ liệu không mang tính đại diện, bị nhiễu hoặc bị ô nhiễm bởi features không liên quan (rác vào, rác ra). Cuối cùng, model của bạn không cần quá đơn giản (trong trường hợp đó nó sẽ không vừa đủ) cũng không quá phức tạp (trong trường hợp đó nó sẽ quá vừa).

Chỉ còn một chủ đề quan trọng cuối cùng cần đề cập: một khi bạn đã training model, bạn không muốn chỉ “hy vọng” nó sẽ khái quát hóa cho các trường hợp mới. Bạn muốn đánh giá nó và hoàn thiện nó nếu cần thiết. Hãy xem cách thực hiện điều đó.

Kiểm tra và xác nhận

Cách duy nhất để biết model sẽ khái quát hóa tốt như thế nào đối với các trường hợp mới là thực sự thử nó trên các trường hợp mới. Một cách để làm điều đó là đưa model của bạn vào sản xuất và theo dõi xem nó hoạt động tốt như thế nào. Điều này hoạt động tốt, nhưng nếu model của bạn quá tệ, user của bạn sẽ phàn nàn—không phải là ý tưởng hay nhất.

Tùy chọn tốt hơn là chia dữ liệu của bạn thành hai bộ: training set và test set. Như những cái tên này ngụ ý, bạn training model của mình bằng training set và bạn kiểm tra nó bằng test set. Tỷ lệ lỗi trong các trường hợp mới được gọi là lỗi tổng quát hóa (hoặc lỗi ngoài mẫu) và bằng cách đánh giá model của bạn trên test set, bạn sẽ ước tính được lỗi này. Giá trị này cho bạn biết model của bạn sẽ hoạt động tốt như thế nào trong các phiên bản mà nó chưa từng thấy trước đây.

Nếu training error thấp (tức là model của bạn mắc ít lỗi trên training set) nhưng lỗi tổng quát hóa cao, điều đó có nghĩa là model của bạn là overfitting dữ liệu training.

Người ta thường sử dụng 80% dữ liệu để training và giữ lại 20% để thử nghiệm. Tuy nhiên, điều này phụ thuộc vào kích thước của dataset: nếu nó chứa 10 triệu phiên bản, thì việc giữ lại 1% có nghĩa là test set của bạn sẽ chứa 100.000 phiên bản, có thể là quá đủ để có được ước tính tốt về lỗi tổng quát hóa.

0.11 Điều chỉnh siêu tham số và lựa chọn model

Việc đánh giá model khá đơn giản: chỉ cần sử dụng test set. Nhưng giả sử bạn đang phân vân giữa hai loại models (chẳng hạn như model tuyến tính và model đa thức): làm thế nào bạn có thể quyết định giữa chúng? Một lựa chọn là training cả hai và so sánh mức độ khái quát hóa của chúng bằng cách sử dụng test set.

Bây giờ giả sử rằng model tuyến tính khái quát hóa tốt hơn, nhưng bạn muốn áp dụng một số regularization để tránh overfitting. Câu hỏi là, làm thế nào để bạn chọn giá trị của regularization hyperparameter? Một lựa chọn là training 100 models khác nhau bằng cách sử dụng 100 giá trị khác nhau cho hyperparameter này. Giả sử bạn tìm thấy giá trị siêu tham số tốt nhất tạo ra model với sai số tổng quát hóa thấp nhất—chẳng hạn, chỉ sai số 5%. Bạn đưa model này vào sản xuất nhưng rất tiếc là nó không hoạt động tốt như mong đợi và gây ra lỗi 15%. Chuyện gì vừa xảy ra vậy?

Vấn đề là bạn đã đo lỗi tổng quát hóa nhiều lần trên test set và bạn đã điều chỉnh model và hyperparameters để tạo ra model tốt nhất cho tập hợp cụ thể đó. Điều này có nghĩa là model khó có thể hoạt động tốt trên dữ liệu mới.

Một giải pháp phổ biến cho vấn đề này được gọi là xác thực giữ lại: bạn chỉ cần giữ một phần training set để đánh giá một số ứng cử viên models và chọn ứng cử viên tốt nhất. Tập được giữ lại mới được gọi là validation set (hoặc đôi khi là tập phát triển hoặc tập phát triển). Cụ thể hơn, bạn training nhiều models với nhiều hyperparameters khác nhau trên training set rút gọn (tức là training set đầy đủ trừ validation set) và bạn chọn model hoạt động tốt nhất trên validation set. Sau quá trình xác thực loại trừ này, bạn training model tốt nhất trên training set đầy đủ (bao gồm cả bộ xác thực) và điều này mang lại cho bạn model cuối cùng. Cuối cùng, bạn đánh giá model cuối cùng này trên test set để ước tính lỗi tổng quát hóa.

Giải pháp này thường hoạt động khá tốt. Tuy nhiên, nếu validation set quá nhỏ thì các đánh giá model sẽ không chính xác: cuối cùng bạn có thể chọn nhầm model dưới mức tối ưu. Ngược lại, nếu validation set quá lớn thì training set còn lại sẽ nhỏ hơn nhiều so với training set đầy đủ. Tại sao điều này lại tệ? Chà, vì model cuối cùng sẽ được training trên training set đầy đủ nên việc so sánh ứng viên models được training trên training set nhỏ hơn nhiều là không lý tưởng. Nó giống như việc chọn người chạy nước rút nhanh nhất để tham gia một cuộc chạy marathon. Một cách để giải quyết vấn đề này là thực hiện lặp lại cross-validation, sử dụng nhiều bộ xác thực nhỏ. Mỗi model được đánh giá một lần cho mỗi validation set sau khi được training trên phần còn lại của dữ liệu. Bằng cách lấy trung bình tất cả

Sau khi đánh giá model, bạn sẽ có được thước đo chính xác hơn nhiều về hiệu suất của nó. Tuy nhiên, có một nhược điểm: thời gian training được nhân với số lượng bộ xác nhận.

0.12 Dữ liệu không khớp

Trong một số trường hợp, thật dễ dàng để có được một lượng lớn dữ liệu cho việc training, nhưng dữ liệu này có thể sẽ không đại diện hoàn hảo cho dữ liệu sẽ được sử dụng trong sản xuất. Ví dụ: giả sử bạn muốn tạo một ứng dụng di động để chụp ảnh hoa và tự động xác định loài của chúng. Bạn có thể dễ dàng tải xuống hàng triệu bức ảnh về hoa trên web nhưng chúng sẽ không đại diện hoàn hảo cho những bức ảnh thực sự được chụp bằng ứng dụng trên thiết bị di động. Có lẽ bạn chỉ có 10.000 bức ảnh đại diện (tức là được chụp thực sự bằng ứng dụng). Trong trường hợp này, quy tắc quan trọng nhất cần nhớ là validation set và test set phải mang tính đại diện nhất có thể cho dữ liệu bạn muốn sử dụng trong sản xuất, do đó, chúng chỉ nên bao gồm các ảnh đại diện: bạn có thể xáo trộn chúng và đặt một nửa vào bộ xác thực và một nửa trong test set (đảm bảo rằng không có bản sao hoặc gần trùng lặp nào xuất hiện trong cả hai bộ). Nhưng sau khi training model của bạn trên các hình ảnh trên web, nếu bạn nhận thấy rằng hiệu suất của model trên validation set thật đáng thất vọng, bạn sẽ không biết liệu điều này có phải là do model của bạn đã quá phù hợp với training set hay đây chỉ là do sự không khớp giữa hình ảnh trên web và hình ảnh trong ứng dụng di động. Một giải pháp là đưa ra một số hình ảnh training (từ web) trong một bộ khác mà Andrew Ng gọi là bộ train-dev. Sau khi model được training (trên training set, không phải trên tập train-dev), bạn có thể đánh giá nó trên tập train-dev. Nếu nó hoạt động tốt thì model không phải là overfitting mà là training set. Nếu nó hoạt động kém trên validation set thì vấn đề chắc chắn là do dữ liệu không khớp. Bạn có thể thử giải quyết vấn đề này bằng cách xử lý trước hình ảnh trên web để làm cho chúng trông giống ảnh sẽ được ứng dụng di động chụp hơn, sau đó training lại model. Ngược lại, nếu model hoạt động kém trên tập train-dev thì chắc chắn nó phải quá khớp với training set, vì vậy bạn nên cố gắng đơn giản hóa hoặc chuẩn hóa model, lấy thêm dữ liệu training và làm sạch dữ liệu training.

0.13 Định lý không có bữa trưa miễn phí

Model là phiên bản đơn giản hóa của các quan sát. Việc đơn giản hóa nhằm loại bỏ các chi tiết thừa không có khả năng khái quát hóa cho các trường hợp mới. Để quyết định dữ liệu nào cần loại bỏ và dữ liệu nào cần giữ lại, bạn phải đưa ra các giả định. Ví dụ: model tuyến tính đưa ra giả định rằng dữ liệu về cơ bản là tuyến tính và khoảng cách giữa các phiên bản và đường thẳng chỉ là nhiễu, có thể bỏ qua một cách an toàn.

Trong một bài báo nổi tiếng năm 1996,¹¹ David Wolpert đã chứng minh rằng nếu bạn hoàn toàn không đưa ra giả định nào về dữ liệu thì không có lý do gì để thích một model hơn bất kỳ model nào khác. Đây được gọi là định lý Không ăn trưa miễn phí (NFL). Đối với một số datasets, model tốt nhất là model tuyến tính, trong khi đối với datasets khác thì đó là neural network. Không có model nào được đảm bảo hoạt động tốt hơn (do đó có tên của định lý). Cách duy nhất để biết chắc chắn model nào là tốt nhất là đánh giá tất cả. Vì điều này là không thể nên trong thực tế, bạn đưa ra một số giả định hợp lý về dữ liệu và chỉ đánh giá một số models hợp lý. Ví dụ: đối với các nhiệm vụ đơn giản, bạn có thể đánh giá models tuyến tính với nhiều cấp độ regularization khác nhau và đối với một vấn đề phức tạp, bạn có thể đánh giá nhiều neural networks khác nhau.

1 Bài tập

Trong chương này chúng ta đã đề cập đến một số khái niệm quan trọng nhất trong Machine Learning. Trong các chương tiếp theo, chúng ta sẽ tìm hiểu sâu hơn và viết nhiều code hơn, nhưng trước khi thực hiện, hãy đảm bảo rằng bạn biết cách trả lời các câu hỏi sau:

1. Bạn định nghĩa Machine Learning như thế nào?
 2. Bạn có thể kể tên bốn loại vấn đề mà nó bọc lộ không?
 3. Training set được gắn nhãn là gì?
 4. Hai nhiệm vụ supervised phổ biến nhất là gì?
 5. Bạn có thể kể tên bốn nhiệm vụ không supervised phổ biến không?
 6. Bạn sẽ sử dụng loại thuật toán Machine Learning nào để cho phép robot đi bộ ở nhiều địa hình không xác định khác nhau?
 7. Bạn sẽ sử dụng loại thuật toán nào để phân chia khách hàng của mình thành nhiều nhóm?
 8. Bạn sẽ coi vấn đề phát hiện thư rác là vấn đề supervised learning hay vấn đề unsupervised learning?
 9. Hệ thống học trực tuyến là gì?
 10. Học tập ngoài cốt lõi là gì?
 11. Loại thuật toán học nào dựa vào thước đo độ tương tự để đưa ra dự đoán?
 12. Sự khác biệt giữa model parameter và hyperparameter của thuật toán learning là gì?
 13. Thuật toán learning dựa trên model tìm kiếm điều gì? Chiến lược phổ biến nhất họ sử dụng để thành công là gì? Họ đưa ra dự đoán như thế nào?
 14. Bạn có thể kể tên bốn thử thách chính trong Machine Learning không?
 15. Nếu model của bạn hoạt động tốt trên dữ liệu training nhưng có khả năng khái quát kém đối với các phiên bản mới, điều gì đang xảy ra? Bạn có thể kể tên ba giải pháp khả thi không?
 16. Test set là gì và tại sao bạn muốn sử dụng nó?
 17. Mục đích của validation set là gì?
 18. Train-dev set là gì, khi nào bạn cần nó và bạn sử dụng nó như thế nào?
 19. Điều gì có thể xảy ra nếu bạn điều chỉnh hyperparameters bằng test set?
- Lời giải cho các bài tập này có sẵn trong Phụ lục A.

1.1 Chương 2: Dự án Machine Learning từ đầu đến cuối

Trong chương này, bạn sẽ làm việc thông qua một dự án mẫu từ đầu đến cuối, giả vờ là một nhà khoa học dữ liệu mới được thuê tại một công ty bất động sản. Dưới đây là các bước chính bạn sẽ trải qua:

1. Nhìn vào bức tranh lớn.
2. Lấy dữ liệu.
3. Khám phá và trực quan hóa dữ liệu để hiểu rõ hơn.
4. Chuẩn bị dữ liệu cho thuật toán Machine Learning.
5. Chọn một model và training nó.
6. Tuning model của bạn.
7. Trình bày giải pháp của bạn.
8. Khởi chạy, giám sát và bảo trì hệ thống của bạn.

1.2 Làm việc với dữ liệu thực

Khi bạn đang tìm hiểu về Machine Learning, tốt nhất bạn nên thử nghiệm với dữ liệu trong thế giới thực chứ không phải datasets nhân tạo. May mắn thay, có hàng nghìn datasets mở để bạn lựa chọn, trải rộng trên tất cả các loại miền. Dưới đây là một số nơi bạn có thể tìm để lấy dữ liệu:

- Các kho dữ liệu mở phổ biến
 - Kho lưu trữ UC Irvine Machine Learning
 - Kaggle datasets
 - AWS datasets của Amazon
- Cổng meta (liệt kê các kho dữ liệu mở)
 - Cổng dữ liệu
 - OpenDataMonitor
 - Quandl
- Các trang khác liệt kê nhiều kho dữ liệu mở phổ biến
 - Danh sách Machine Learning datasets trên Wikipedia
 - Quora.com
 - Subreddit datasets

Trong chương này, chúng ta sẽ sử dụng dataset giá nhà ở California từ kho lưu trữ StatLib. Dataset này dựa trên dữ liệu từ điều tra dân số California năm 1990. Nó không phải là dữ liệu mới nhất (một căn nhà ở khu vực Bay Area vẫn còn có thể mua được tại thời điểm đó), nhưng nó có nhiều tính chất tốt cho việc học, vì vậy chúng ta sẽ giả sử nó là dữ liệu mới nhất. Để mục đích giảng dạy, tôi đã thêm một thuộc tính loại và loại bỏ một số thuộc tính.

1.3 Nhìn vào bức tranh lớn

Chào mừng đến với Tập đoàn Nhà ở Machine Learning! Nhiệm vụ đầu tiên của bạn là sử dụng dữ liệu điều tra dân số California để xây dựng model về giá nhà đất trong tiểu bang. Dữ liệu này bao gồm các số liệu như dân số, thu nhập trung bình và giá nhà trung bình cho từng nhóm khối ở California. Các nhóm khối là đơn vị địa lý nhỏ nhất mà Cục điều tra dân số US công bố dữ liệu mẫu (một nhóm khối thường có dân số từ 600 đến 3.000 người). Chúng ta sẽ gọi ngắn gọn là “quận”.

Model của bạn sẽ học hỏi từ dữ liệu này và có thể dự đoán giá nhà ở trung bình ở bất kỳ quận nào, dựa trên tất cả các số liệu khác.

Vì bạn là một nhà khoa học dữ liệu có tổ chức tốt nên điều đầu tiên bạn nên làm là lấy danh sách kiểm tra dự án Machine Learning của mình. Bạn có thể bắt đầu với phụ lục B; nó sẽ hoạt động khá tốt cho hầu hết các dự án Machine Learning, nhưng hãy đảm bảo điều chỉnh nó cho phù hợp với nhu cầu của bạn. Trong chương này, chúng ta sẽ điếm qua nhiều mục trong danh sách kiểm tra, nhưng chúng ta cũng sẽ bỏ qua một số mục, vì chúng dễ hiểu hoặc vì chúng sẽ được thảo luận trong các chương sau.

1.4 Đóng khung vấn đề

Câu hỏi đầu tiên bạn nên hỏi sắp là mục tiêu kinh doanh chính xác là gì. Xây dựng model có lẽ không phải là mục tiêu cuối cùng. Công ty mong đợi sử dụng và hưởng lợi như thế nào từ model này? Biết mục tiêu là quan trọng vì nó sẽ quyết định cách bạn đặt vấn đề,

thuật toán nào bạn sẽ chọn, thước đo hiệu suất nào bạn sẽ sử dụng để đánh giá model của mình và bạn sẽ bỏ ra bao nhiêu công sức để điều chỉnh nó.

Sếp của bạn trả lời rằng đầu ra model của bạn (dự đoán về giá nhà trung bình của một quận) sẽ được cung cấp cho một hệ thống Machine Learning khác, cùng với nhiều tín hiệu khác. Hệ thống xuôi dòng này sẽ xác định liệu nó có đáng đầu tư vào một khu vực nhất định hay không. Việc thực hiện đúng điều này là rất quan trọng vì nó ảnh hưởng trực tiếp đến doanh thu.

1.5 Đường ống

Một chuỗi các thành phần xử lý dữ liệu được gọi là dữ liệu pipeline. Pipelines rất phổ biến trong các hệ thống Machine Learning, vì có rất nhiều dữ liệu cần thao tác và nhiều phép chuyển đổi dữ liệu cần áp dụng.

Các thành phần thường chạy không đồng bộ. Mỗi thành phần lấy một lượng lớn dữ liệu, xử lý nó và đưa ra kết quả trong một kho dữ liệu khác. Sau đó, một thời gian sau, thành phần tiếp theo trong pipeline lấy dữ liệu này và đưa ra đầu ra của chính nó. Mỗi thành phần khá khép kín: giao diện giữa các thành phần chỉ đơn giản là nơi lưu trữ dữ liệu. Điều này làm cho hệ thống trở nên dễ nắm bắt (với sự trợ giúp của biểu đồ luồng dữ liệu) và các nhóm khác nhau có thể tập trung vào các thành phần khác nhau. Hơn nữa, nếu một thành phần bị hỏng, các thành phần phía sau thường có thể tiếp tục chạy bình thường (ít nhất là trong một thời gian) chỉ bằng cách sử dụng đầu ra cuối cùng từ thành phần bị hỏng. Điều này làm cho kiến trúc khá mạnh mẽ.

Mặt khác, một bộ phận bị hỏng có thể không được chú ý trong một thời gian nếu không thực hiện giám sát thích hợp. Dữ liệu trở nên cũ và hiệu suất của toàn bộ hệ thống giảm xuống.

Câu hỏi tiếp theo bạn nên hỏi sếp là giải pháp hiện tại trông như thế nào (nếu có). Tình huống hiện tại thường sẽ cung cấp cho bạn thông tin tham khảo về hiệu suất cũng như thông tin chuyên sâu về cách giải quyết vấn đề. Sếp của bạn trả lời rằng giá nhà ở trong quận hiện được các chuyên gia ước tính thủ công: một nhóm thu thập thông tin cập nhật về quận và khi họ không thể có được giá nhà ở trung bình, họ sẽ ước tính giá đó bằng cách sử dụng các quy tắc phức tạp.

Việc này tốn kém và mất thời gian, và ước tính của họ không lớn; trong trường hợp họ cố gắng tìm ra giá nhà ở trung bình thực tế, họ thường nhận ra rằng ước tính của họ đã sai lệch hơn 20%. Đây là lý do tại sao công ty cho rằng sẽ hữu ích nếu training model để dự đoán giá nhà ở trung bình của một quận, trong điều kiện khác.

Dữ liệu về quận đó. Dữ liệu điều tra dân số trông giống như một dataset tuyệt vời để khai thác cho mục đích này vì nó bao gồm giá nhà ở trung bình của hàng nghìn quận cũng như các dữ liệu khác.

Với tất cả thông tin này, bạn đã sẵn sàng bắt đầu thiết kế hệ thống của mình. Trước tiên, bạn cần xác định vấn đề: nó supervised, không supervised hay Reinforcement Learning? Đó có phải là nhiệm vụ classification, nhiệm vụ regression hay nhiệm vụ nào khác không? Bạn nên sử dụng kỹ thuật học batch hay học trực tuyến? Trước khi đọc tiếp, hãy tạm dừng và cố gắng tự trả lời những câu hỏi này.

Bạn đã tìm thấy câu trả lời chưa? Hãy xem: đây rõ ràng là một nhiệm vụ supervised learning điển hình, vì bạn được gắn nhãn training examples (mỗi trường hợp đều đi kèm với sản lượng dự kiến, tức là giá nhà ở trung bình của quận). Đây cũng là một nhiệm vụ regression điển hình vì bạn được yêu cầu dự đoán một giá trị. Cụ thể hơn, đây là bài toán bội regression, vì hệ thống sẽ sử dụng nhiều features để đưa ra dự đoán (nó sẽ sử

dung dân số của quận, thu nhập trung bình, v.v.). Đây cũng là bài toán regression một biến, vì chúng ta chỉ cố gắng dự đoán một giá trị duy nhất cho mỗi quận. Nếu chúng ta cố gắng dự đoán nhiều giá trị cho mỗi quận thì đó sẽ là vấn đề regression đa biến. Cuối cùng, không có luồng dữ liệu liên tục đi vào hệ thống, không có nhu cầu cụ thể nào để điều chỉnh để thay đổi dữ liệu nhanh chóng và dữ liệu đủ nhỏ để vừa trong bộ nhớ, vì vậy việc học batch đơn giản sẽ hoạt động tốt.

Nếu dữ liệu lớn, bạn có thể chia công việc học batch của mình trên nhiều máy chủ (sử dụng kỹ thuật MapReduce) hoặc sử dụng kỹ thuật học trực tuyến.

1.6 Chọn thước đo hiệu suất

Bước tiếp theo của bạn là chọn thước đo hiệu suất. Một thước đo hiệu suất điển hình cho các vấn đề regression là Lỗi bình phương trung bình gốc (RMSE). Nó đưa ra ý tưởng về mức độ lỗi mà hệ thống thường mắc phải trong các dự đoán của nó, với weight cao hơn đối với các lỗi lớn.

1.7 Ký hiệu

Phương trình này giới thiệu một số ký hiệu Machine Learning rất phổ biến mà chúng ta sẽ sử dụng trong suốt cuốn sách này:

- m là số phiên bản trong dataset mà bạn đang đo RMSE trên đó.
 - Ví dụ: nếu bạn đang đánh giá RMSE trên validation set gồm 2.000 quận thì $m = 2.000$.
- $x(i)$ là một vector của tất cả các giá trị feature (không bao gồm nhãn) của phiên bản thứ i trong dataset, và $y(i)$ là nhãn của nó (giá trị đầu ra mong muốn cho phiên bản đó).
 - Ví dụ: nếu quận đầu tiên trong dataset nằm ở kinh độ $-118,29^\circ$, vĩ độ $33,91^\circ$ và có 1.416 cư dân với thu nhập trung bình là 38,372, và giá trung bình của ngôi nhà là 156,400 (bỏ qua features khác ở thời điểm hiện tại), thì:
 - Ví dụ: nếu quận đầu tiên như mô tả thì matrix X trông như thế này:
- h là hàm dự đoán của hệ thống, còn được gọi là giả thuyết. Khi hệ thống của bạn được cung cấp vector feature $x(i)$ của một phiên bản, nó sẽ xuất ra một giá trị dự đoán $\hat{y}(i) = h(x(i))$ cho phiên bản đó (ý được phát âm là “y-hat”).
 - Ví dụ: nếu hệ thống của bạn dự đoán rằng giá nhà ở trung bình ở quận đầu tiên là \$158.400, thì $\hat{y}(1) = h(x(1)) = 158.400$. Sai số dự đoán cho quận này là $\hat{y}(1) - y(1) = 2.000$.
- $RMSE(X, h)$ là hàm chi phí được đo trên tập hợp các ví dụ sử dụng giả thuyết h của bạn.

Mặc dù RMSE thường là thước đo hiệu suất ưa thích cho các tác vụ regression, nhưng trong một số trường hợp, bạn có thể thích sử dụng chức năng khác hơn. Ví dụ: giả sử có nhiều quận ngoại lệ. Trong trường hợp đó, bạn có thể cân nhắc sử dụng sai số tuyệt đối trung bình (MAE, còn được gọi là độ lệch tuyệt đối trung bình).

Cả RMSE và MAE đều là những cách để đo khoảng cách giữa hai vector: vector dự đoán và vector giá trị đích. Có thể có nhiều thước đo khoảng cách hoặc định mức khác nhau:

- Tính căn bậc hai của tổng bình phương (RMSE) tương ứng với chuẩn Euclidean: đây là khái niệm về khoảng cách mà bạn quen thuộc. Nó còn được gọi là chuẩn 2.

- Tính tổng các giá trị tuyệt đối (MAE) tương ứng với chỉ tiêu 1. Điều này đôi khi được gọi là tiêu chuẩn Manhattan vì nó đo khoảng cách giữa hai điểm trong thành phố nếu bạn chỉ có thể di chuyển dọc theo các khối thành phố trực giao.

- Tổng quát hơn, chuẩn k của vector v chứa n phần tử được định nghĩa là $\sqrt[k]{\sum_{i=1}^n |v_i|^k}$. Cho biết số phần tử khác 0 trong vector và ∞ cho giá trị tuyệt đối lớn nhất trong vector.

- Chỉ số định mức càng cao thì càng tập trung vào những giá trị lớn và bỏ qua những giá trị nhỏ. Đây là lý do tại sao RMSE nhạy cảm hơn với các giá trị ngoại lệ so với MAE. Nhưng khi các ngoại lệ hiếm gặp theo cấp số nhân (như trong đường cong hình chuông), RMSE hoạt động rất tốt và thường được ưa thích hơn.

1.8 Kiểm tra các giả định

Cuối cùng, cách tốt nhất là liệt kê và xác minh các giả định đã được đưa ra cho đến nay (bởi bạn hoặc người khác); điều này có thể giúp bạn sớm phát hiện được các vấn đề nghiêm trọng. Ví dụ: giá quận mà hệ thống của bạn đưa ra sẽ được đưa vào hệ thống Machine Learning xuôi dòng và bạn giả định rằng những giá này sẽ được sử dụng như vậy. Nhưng điều gì sẽ xảy ra nếu hệ thống tiếp theo chuyển đổi giá thành các loại (ví dụ: “rẻ”, “trung bình” hoặc “đắt”) và sau đó sử dụng các loại đó thay vì chính giá? Trong trường hợp này, việc có được mức giá hoàn toàn phù hợp không hề quan trọng chút nào; hệ thống của bạn chỉ cần classification đúng. Nếu đúng như vậy thì vấn đề đáng lẽ phải được đóng khung dưới dạng nhiệm vụ classification chứ không phải nhiệm vụ regression. Bạn không muốn phát hiện ra điều này sau khi làm việc trên hệ thống regression trong nhiều tháng.

May mắn thay, sau khi nói chuyện với nhóm phụ trách hệ thống hạ nguồn, bạn tin tưởng rằng họ thực sự cần giá thực tế chứ không chỉ danh mục. Tuyệt vời! Bạn đã hoàn tất, đèn xanh và bạn có thể bắt đầu viết code ngay bây giờ!

1.9 Lấy dữ liệu

Đã đến lúc bạn phải làm bản tay mình. Đừng ngần ngại cầm máy tính xách tay của bạn lên và xem qua các ví dụ về code sau trong Jupyter notebook. Sổ ghi chú Jupyter đầy đủ có sẵn tại <https://github.com/ageron/handson-ml2>.

1.10 Tạo không gian làm việc

Trước tiên, bạn cần cài đặt Python. Nó có thể đã được cài đặt trên hệ thống của bạn. Nếu không, bạn có thể lấy nó tại <https://www.python.org/>.

Tiếp theo, bạn cần tạo một thư mục không gian làm việc cho code Machine Learning và datasets của mình. Mở terminal và gõ các lệnh sau (sau dấu nhắc \$):

Bạn sẽ cần một số mô-đun Python: Jupyter, NumPy, pandas, Matplotlib và Scikit-Learn. Nếu bạn đã cài đặt Jupyter với tất cả các mô-đun này, bạn có thể chuyển sang phần “Tải xuống dữ liệu” trên trang 46 một cách an toàn. Nếu bạn chưa có chúng, có nhiều cách để cài đặt chúng (và các phần phụ thuộc của chúng). Bạn có thể sử dụng hệ thống đóng gói của hệ thống (ví dụ: apt-get trên Ubuntu hoặc MacPorts hoặc Homebrew trên macOS), cài đặt bản phân phối Scientific Python như Anaconda và sử dụng hệ thống đóng gói của nó hoặc chỉ sử dụng hệ thống đóng gói của riêng Python, pip, được bao gồm bởi

Mặc định với trình cài đặt nhị phân Python (kể từ Python 2.7.9).

Bạn nên đảm bảo rằng bạn đã cài đặt phiên bản pip gần đây. Để nâng cấp mô-đun pip, hãy nhập thông tin sau (phiên bản chính xác có thể khác).

1.11 Tạo một môi trường biệt lập

Nếu bạn muốn làm việc trong một môi trường biệt lập (được khuyến nghị mạnh mẽ để bạn có thể làm việc trên các dự án khác nhau mà không có các phiên bản thư viện xung đột), hãy cài đặt virtualenv8 bằng cách chạy lệnh pip sau (một lần nữa, nếu bạn muốn cài đặt virtualenv cho tất cả user trên máy của mình, hãy xóa `-user` và chạy lệnh này với quyền quản trị viên).

Bây giờ bạn có thể tạo môi trường Python bị cô lập.

Tôi sẽ hiển thị các bước cài đặt bằng pip trong bash shell trên hệ thống Linux hoặc macOS. Bạn có thể cần phải điều chỉnh các lệnh này cho phù hợp với hệ thống của mình. Trên Windows, tôi khuyên bạn nên cài đặt Anaconda.

Để hủy kích hoạt môi trường này, hãy nhập hủy kích hoạt. Khi môi trường đang hoạt động, mọi gói bạn cài đặt bằng pip sẽ được cài đặt trong môi trường biệt lập này và Python sẽ chỉ có quyền truy cập vào các gói này (nếu bạn cũng muốn truy cập vào các gói của hệ thống, bạn nên tạo môi trường bằng tùy chọn `-system-sitepackages` của virtualenv). Kiểm tra tài liệu của virtualenv để biết thêm thông tin.

Bây giờ bạn có thể cài đặt tất cả các mô-đun cần thiết và các phần phụ thuộc của chúng bằng cách sử dụng lệnh pip đơn giản này (nếu bạn không sử dụng virtualenv, bạn sẽ cần tùy chọn `-user` hoặc quyền quản trị viên).

Nếu bạn đã tạo một virtualenv, bạn cần đăng ký nó với Jupyter và đặt tên cho nó.

Bây giờ bạn có thể kích hoạt Jupyter.

Máy chủ Jupyter hiện đang chạy trong thiết bị đầu cuối của bạn, nghe cổng 8888. Bạn có thể truy cập máy chủ này bằng cách mở trình duyệt web của mình tới <http://localhost:8888/> (điều này thường tự động xảy ra khi máy chủ khởi động). Bạn sẽ thấy thư mục không gian làm việc trống (chỉ chứa thư mục `env` nếu bạn làm theo hướng dẫn virtualenv trước đó).

Bây giờ, hãy tạo Python notebook mới bằng cách nhấp vào nút Mới và chọn phiên bản Python9 thích hợp (xem Hình 2-3). Làm như vậy sẽ tạo một tệp notebook mới có tên `Untitled.ipynb` trong không gian làm việc của bạn, khởi động kernel Jupyter Python để chạy notebook và mở notebook này trong một tab mới. Bạn nên bắt đầu bằng cách đổi tên notebook này thành “Housing” (điều này sẽ tự động đổi tên tệp thành `Housing.ipynb`) bằng cách nhấp vào Chưa có tiêu đề và nhập tên mới.

1.12 Tải xuống dữ liệu

Trong các môi trường thông thường, dữ liệu của bạn sẽ có sẵn trong cơ sở dữ liệu quan hệ (hoặc một số kho lưu trữ dữ liệu phổ biến khác) và trải rộng trên nhiều bảng/tài liệu/tệp. Để truy cập nó, trước tiên bạn cần có thông tin xác thực và ủy quyền truy cập¹⁰ cũng như làm quen với lược đồ dữ liệu. Tuy nhiên, trong dự án này, mọi thứ đơn giản hơn nhiều: bạn sẽ chỉ tải xuống một tệp nén duy nhất, `housing.tgz`, chứa tệp có giá trị được phân tách bằng dấu phẩy (CSV) có tên là `housing.csv` cùng với tất cả dữ liệu.

Bạn có thể sử dụng trình duyệt web để tải tệp xuống và chạy `tar xzf housing.tgz` để giải nén và giải nén tệp CSV, nhưng tốt nhất bạn nên tạo một chức năng nhỏ để thực hiện việc đó. Việc có một chức năng tải xuống dữ liệu sẽ rất hữu ích nếu dữ liệu thay đổi thường xuyên: bạn có thể viết một tập lệnh nhỏ sử dụng chức năng này để tìm nạp dữ

liệu mới nhất (hoặc bạn có thể thiết lập một công việc đã lên lịch để thực hiện việc đó một cách tự động theo các khoảng thời gian đều đặn). Tự động hóa quá trình tìm nạp dữ liệu cũng hữu ích nếu bạn cần cài đặt dataset trên nhiều máy.

Bây giờ hãy tải dữ liệu bằng pandas. Một lần nữa, bạn nên viết một hàm nhỏ để tải dữ liệu:

Hàm này trả về một đối tượng pandas DataFrame chứa tất cả dữ liệu.

1.13 Xem nhanh cấu trúc dữ liệu

Chúng ta hãy xem năm hàng trên cùng bằng phương thức `head()` của DataFrame.

Phương thức `info()` rất hữu ích để có được mô tả nhanh về dữ liệu, đặc biệt là tổng số hàng, loại của từng thuộc tính và số lượng giá trị không rỗng.

Có 20.640 phiên bản trong dataset, điều đó có nghĩa là nó khá nhỏ so với tiêu chuẩn Machine Learning, nhưng thật hoàn hảo để bắt đầu. Lưu ý rằng thuộc tính `total_bed` chỉ có 20.433 giá trị không rỗng, nghĩa là 207 quận thiếu feature này. Chúng ta sẽ cần phải giải quyết vấn đề này sau.

Hãy nhìn vào các lĩnh vực khác. Phương thức `mô tả()` hiển thị bản tóm tắt các thuộc tính số (Hình 2-7).

Các hàng 25%, 50% và 75% hiển thị các phần trăm tương ứng: một phần trăm biểu thị giá trị dưới đó một phần trăm của các quan sát trong một nhóm quan sát rơi vào. Ví dụ, 25% của các quận có `housing_median_age` thấp hơn 18, trong khi 50% thấp hơn 29 và 75% thấp hơn 37. Những con số này thường được gọi là phần trăm 25 (hoặc số bốn đầu tiên), trung bình và phần trăm 75 (hoặc số bốn thứ ba).

Có một số điều bạn có thể nhận thấy trong các biểu đồ này:

1. Đầu tiên, thuộc tính thu nhập trung bình có vẻ như không được biểu thị bằng đô la US (USD). Sau khi kiểm tra với nhóm đã thu thập dữ liệu, bạn được thông báo rằng dữ liệu đã được chia tỷ lệ và giới hạn ở mức 15 (thực tế là 15,0001) đối với thu nhập trung bình cao hơn và ở mức 0,5 (thực tế là 0,4999) đối với thu nhập trung bình thấp hơn. Các con số đại diện cho khoảng hàng chục nghìn đô la (ví dụ: 3 thực sự có nghĩa là khoảng 30.000 đô la). Làm việc với các thuộc tính được xử lý trước là điều phổ biến trong Machine Learning và nó không hẳn là một vấn đề, nhưng bạn nên cố gắng hiểu cách tính toán dữ liệu.

2. Tuổi trung bình của nhà ở và giá trị nhà trung bình cũng bị giới hạn. Cái sau có thể là một vấn đề nghiêm trọng vì nó là thuộc tính mục tiêu của bạn (nhãn của bạn). Thuật toán Machine Learning của bạn có thể biết rằng giá không bao giờ vượt quá giới hạn đó. Bạn cần kiểm tra với nhóm khách hàng của mình (nhóm sẽ sử dụng đầu ra của hệ thống của bạn) để xem đây có phải là sự cố hay không. Nếu họ nói với bạn rằng họ cần những dự đoán chính xác thậm chí vượt quá 500.000 USD thì bạn có hai lựa chọn:

- A. Thu thập nhãn thích hợp cho các huyện có nhãn bị giới hạn.

3. Những thuộc tính này có quy mô rất khác nhau. Chúng ta sẽ thảo luận vấn đề này sau trong chương này, khi chúng ta khám phá khả năng chia tỷ lệ feature.

4. Cuối cùng, nhiều biểu đồ có phần đuôi nặng: chúng kéo dài về bên phải của đường trung tuyến hơn là về bên trái. Điều này có thể khiến một số thuật toán Machine Learning khó phát hiện các mẫu hơn một chút. Sau này chúng ta sẽ thử chuyển đổi các thuộc tính này để có nhiều phân bố hình chuông hơn.

Hy vọng bây giờ bạn đã hiểu rõ hơn về loại dữ liệu bạn đang xử lý.

Chờ đợi! Trước khi xem xét thêm dữ liệu, bạn cần tạo test set, đặt nó sang một bên và không bao giờ nhìn vào nó.

1.14 Tạo một bộ thử nghiệm

Nghe có vẻ lạ khi tự nguyện dành một phần dữ liệu ở giai đoạn này. Rốt cuộc, bạn chỉ xem qua dữ liệu và chắc chắn bạn nên tìm hiểu thêm nhiều điều về nó trước khi quyết định sử dụng thuật toán nào, phải không? Điều này đúng, nhưng bộ não của bạn là một hệ thống phát hiện khuôn mẫu đáng kinh ngạc, điều đó có nghĩa là nó rất dễ mắc phải *overfitting*: nếu bạn nhìn vào test set, bạn có thể tình cờ phát hiện ra một số khuôn mẫu có vẻ thú vị trong dữ liệu thử nghiệm khiến bạn chọn một loại Machine Learning model cụ thể. Khi bạn ước tính lỗi khái quát hóa bằng test set, ước tính của bạn sẽ quá lạc quan và bạn sẽ khởi chạy một hệ thống không hoạt động tốt như mong đợi. Điều này được gọi là *rình mò dữ liệu bias*.

Về mặt lý thuyết, việc tạo test set rất đơn giản: chọn ngẫu nhiên một số phiên bản, thường là 20% dataset (hoặc ít hơn nếu dataset của bạn rất lớn) và đặt chúng sang một bên:

Chà, điều này hoạt động nhưng nó không hoàn hảo: nếu bạn chạy lại chương trình, nó sẽ tạo ra một test set khác! Theo thời gian, bạn (hoặc thuật toán Machine Learning của bạn) sẽ thấy toàn bộ dataset, đây là điều bạn muốn tránh.

Nhưng cả hai giải pháp này sẽ bị hỏng vào lần tiếp theo bạn tải dataset cập nhật. Để có sự phân chia training/kiểm tra ổn định ngay cả sau khi cập nhật dataset, một giải pháp phổ biến là sử dụng code định danh của từng phiên bản để quyết định xem nó có nên đưa vào test set hay không (giả sử các phiên bản có code định danh duy nhất và không thể thay đổi). Ví dụ: bạn có thể tính toán giá trị băm của code định danh của từng phiên bản và đặt phiên bản đó vào test set nếu giá trị băm thấp hơn hoặc bằng 20% giá trị băm tối đa. Điều này đảm bảo rằng test set sẽ duy trì tính nhất quán trong nhiều lần chạy, ngay cả khi bạn làm mới dataset. Test set mới sẽ chứa 20% phiên bản mới, nhưng nó sẽ không chứa bất kỳ phiên bản nào trước đây có trong training set.

Rất tiếc là housing dataset không có cột nhận dạng. Giải pháp đơn giản nhất là sử dụng chỉ mục hàng làm ID:

Cho đến nay chúng ta đã xem xét các phương pháp lấy mẫu ngẫu nhiên. Điều này thường là tốt nếu dataset của bạn đủ lớn (đặc biệt so với số lượng thuộc tính), nhưng nếu không, bạn có thể chịu rủi ro nhập một sai lệch lấy mẫu đáng kể. Khi một công ty khảo sát quyết định gọi 1,000 người để hỏi họ vài câu hỏi, họ không chỉ chọn 1,000 người ngẫu nhiên trong sổ điện thoại. Họ cố gắng đảm bảo rằng 1,000 người này đại diện cho toàn bộ quần thể. Ví dụ: quần thể của Mỹ là 51,3% nữ và 48,7% nam, vì vậy một khảo sát được thực hiện tốt ở Mỹ sẽ cố gắng duy trì tỷ lệ này trong mẫu: 513 nữ và 487 nam. Đây được gọi là lấy mẫu theo tầng: quần thể được chia thành các nhóm con đồng nhất gọi là tầng, và số lượng mẫu phù hợp được lấy từ mỗi tầng để đảm bảo rằng tập test đại diện cho toàn bộ quần thể. Nếu những người chạy khảo sát sử dụng lấy mẫu ngẫu nhiên, thì có khoảng 12% khả năng lấy mẫu một tập test bị lệch

Đó là ít hơn 49% nữ hoặc hơn 54% nữ. Dù bằng cách nào, kết quả khảo sát sẽ bị sai lệch đáng kể.

Giả sử bạn đã nói chuyện với các chuyên gia đã nói với bạn rằng thu nhập trung bình là một thuộc tính rất quan trọng để dự đoán giá nhà trung bình. Bạn có thể muốn đảm bảo rằng tập test đại diện cho các loại thu nhập khác nhau trong toàn bộ dataset. Vì thu nhập trung bình là một thuộc tính số liên tục, bạn đầu tiên cần tạo một thuộc tính loại thu nhập. Hãy xem xét biểu đồ thu nhập trung bình một cách chi tiết hơn (trở lại Hình 2-8): hầu hết các giá trị thu nhập trung bình nằm trong khoảng 1,5 đến 6 (tức là 15,000–60,000), nhưng một số thu nhập trung bình vượt quá 6. Điều quan trọng là có số

lượng mẫu đủ lớn trong dataset của bạn cho mỗi tầng, nếu không, ước tính về tầng quan trọng có thể bị lệch. Điều này có nghĩa là bạn không nên có quá nhiều tầng, và mỗi tầng nên đủ lớn. Code sau sử dụng hàm `pd.cut()` để tạo một thuộc tính loại thu nhập với năm loại (được đánh số từ 1 đến 5): loại 1 nằm trong khoảng 0 đến 1,5 (tức là ít hơn \$15,000), loại 2 từ 1,5 đến 3, và cứ tiếp tục như vậy:

Các loại thu nhập này được thể hiện trong Hình 2-9:

```
Nhà ở["income_cat"].hist()
```

Bây giờ bạn đã sẵn sàng thực hiện lấy mẫu phân tầng dựa trên loại thu nhập. Để làm điều này, bạn có thể sử dụng lớp `StratifiedShuffleSplit` của `Scikit-Learn`:

Hãy xem liệu điều này có hoạt động như mong đợi không. Bạn có thể bắt đầu bằng cách xem tỷ lệ danh mục thu nhập trong test set:

Với code tương tự, bạn có thể đo lường tỷ lệ danh mục thu nhập trong dataset đầy đủ. Hình 2-10 so sánh tỷ lệ loại thu nhập trong dataset tổng thể, trong test set được tạo bằng lấy mẫu phân tầng và trong test set được tạo bằng cách lấy mẫu hoàn toàn ngẫu nhiên. Như bạn có thể thấy, test set được tạo bằng cách lấy mẫu phân tầng có tỷ lệ danh mục thu nhập gần như giống hệt với dataset đầy đủ, trong khi test set được tạo bằng cách lấy mẫu hoàn toàn ngẫu nhiên bị sai lệch.

Bây giờ bạn nên xóa thuộc tính `income_cat` để dữ liệu trở về trạng thái ban đầu:

Chúng ta đã dành khá nhiều thời gian cho thể hệ test set vì một lý do chính đáng: đây là phần thường bị bỏ qua nhưng quan trọng của dự án Machine Learning. Hơn nữa, nhiều ý tưởng trong số này sẽ hữu ích sau này khi chúng ta thảo luận về cross-validation. Bây giờ là lúc chuyển sang giai đoạn tiếp theo: khám phá dữ liệu.

Khám phá và trực quan hóa dữ liệu để hiểu rõ hơn

Cho đến nay, bạn chỉ lướt qua dữ liệu để hiểu chung về loại dữ liệu bạn đang thao tác. Bây giờ mục tiêu là đi sâu hơn một chút.

Trước tiên, hãy đảm bảo rằng bạn đã đặt test set sang một bên và bạn chỉ đang khám phá tập training. Ngoài ra, nếu training set rất lớn, bạn có thể muốn lấy mẫu một bộ khám phá để thực hiện các thao tác dễ dàng và nhanh chóng. Trong trường hợp của chúng ta, bộ này khá nhỏ nên bạn chỉ có thể làm việc trực tiếp trên bộ đầy đủ. Hãy tạo một bản sao để bạn có thể chơi với nó mà không làm hại training set:

```
Nhà ở = strat_train_set.copy()
```

1.15 Trực quan hóa dữ liệu địa lý

Vì có thông tin địa lý (vĩ độ và kinh độ), nên tạo biểu đồ phân tán của tất cả các quận để trực quan hóa dữ liệu (Hình 2-11):

Điều này trông giống như California, nhưng ngoài điều đó ra thì khó có thể thấy bất kỳ hình mẫu cụ thể nào. Đặt tùy chọn `alpha` thành 0,1 giúp dễ dàng hình dung hơn những nơi có mật độ điểm dữ liệu cao (Hình 2-12):

Bây giờ thì tốt hơn nhiều: bạn có thể thấy rõ các khu vực có mật độ dân số cao, cụ thể là Vùng Vịnh và xung quanh Los Angeles và San Diego, cùng với một hàng dài có mật độ khá cao ở Thung lũng Trung tâm, đặc biệt là xung quanh Sacramento và Fresno.

Bộ não của chúng ta rất giỏi trong việc phát hiện các mẫu trong hình ảnh, nhưng bạn có thể cần thử nghiệm với parameters trực quan để làm nổi bật các mẫu.

Bây giờ chúng ta hãy nhìn vào giá nhà đất (Hình 2-13). Bán kính của mỗi vòng tròn biểu thị dân số của quận (tùy chọn `s`) và màu sắc biểu thị giá cả (tùy chọn `c`). Chúng ta sẽ sử dụng bản đồ màu được xác định trước (tùy chọn `cmap`) được gọi là máy bay phản lực, có phạm vi từ màu xanh lam (giá trị thấp) đến màu đỏ (giá cao)

Hình ảnh này cho bạn biết rằng giá nhà đất liên quan rất nhiều đến vị trí (ví dụ: gần biển) và mật độ dân số, như bạn có thể đã biết. Thuật toán clustering sẽ hữu ích trong việc phát hiện cụm chính và để thêm features mới để đo khoảng cách với các trung tâm cụm. Thuộc tính gần biển cũng có thể hữu ích, mặc dù ở Bắc California, giá nhà ở các quận ven biển không quá cao nên đây không phải là một quy tắc đơn giản.

1.16 Tìm kiếm mối tương quan

Vì dataset không quá lớn nên bạn có thể dễ dàng tính hệ số tương quan tiêu chuẩn (còn gọi là Pearson's r) giữa mỗi cặp thuộc tính bằng phương thức `corr()`:

Bây giờ hãy xem mỗi thuộc tính tương quan với giá trị ngôi nhà trung bình như thế nào:

```
Hộ gia đình 0,064702 total_bedrooms 0,047865 dân số -0,026699 kinh độ -0,047279 vĩ độ -0,142826 Tên: median_house_value, dtype: float64
```

Hệ số tương quan nằm trong khoảng từ -1 đến 1 . Khi nó gần bằng 1 , điều đó có nghĩa là có mối tương quan dương mạnh mẽ; ví dụ, giá trị ngôi nhà trung bình có xu hướng tăng lên khi thu nhập trung bình tăng lên. Khi hệ số gần bằng -1 nghĩa là có mối tương quan nghịch mạnh; bạn có thể thấy mối tương quan nghịch nhỏ giữa vĩ độ và giá trị ngôi nhà trung bình (tức là giá có xu hướng giảm nhẹ khi bạn đi về phía bắc). Cuối cùng, hệ số gần bằng 0 có nghĩa là không có mối tương quan tuyến tính. Hình 2-14 cho thấy các đồ thị khác nhau cùng với hệ số tương quan giữa trục ngang và trục dọc của chúng.

Hệ số tương quan chỉ đo lường mối tương quan tuyến tính (“nếu x tăng thì y thường tăng/giảm”). Nó có thể hoàn toàn bỏ sót các mối quan hệ phi tuyến tính (ví dụ: “nếu x gần bằng 0 thì y thường tăng lên”). Lưu ý rằng tất cả các đồ thị ở hàng dưới cùng đều có hệ số tương quan bằng 0 , mặc dù thực tế là các trục của chúng rõ ràng không độc lập: đây là những ví dụ về mối quan hệ phi tuyến tính. Ngoài ra, hàng thứ hai hiển thị các ví dụ trong đó hệ số tương quan bằng 1 hoặc -1 ; lưu ý rằng điều này không liên quan gì đến gradient. Ví dụ: chiều cao của bạn tính bằng inch có hệ số tương quan là 1 với chiều cao tính bằng feet hoặc tính bằng nanomet.

Một cách khác để kiểm tra mối tương quan giữa các thuộc tính là sử dụng hàm `pandas.scatter_matrix()`, hàm này vẽ đồ thị mọi thuộc tính số đối với mọi thuộc tính.

Thuộc tính số khác. Vì hiện tại có 11 thuộc tính bằng số, bạn sẽ nhận được $11 \times 11 = 121$ ô, không vừa trên một trang—vì vậy, hãy tập trung vào một số thuộc tính đầy hứa hẹn có vẻ tương quan nhất với giá trị nhà ở trung bình (Hình 2-15):

Đường chéo chính (trên cùng bên trái đến dưới cùng bên phải) sẽ có đầy các đường thẳng nếu `pandas` vẽ từng biến theo chính nó, điều này sẽ không hữu ích lắm. Vì vậy, thay vào đó `pandas` hiển thị biểu đồ của từng thuộc tính (có sẵn các tùy chọn khác; xem tài liệu `pandas` để biết thêm chi tiết).

Thuộc tính hứa hẹn nhất để dự đoán giá trị ngôi nhà trung bình là thu nhập trung bình, vì vậy hãy phóng to biểu đồ tương quan của chúng (Hình 2-16):

Cốt truyện này tiết lộ một vài điều. Đầu tiên, mối tương quan thực sự rất mạnh mẽ; bạn có thể thấy rõ xu hướng đi lên và các điểm không quá phân tán. Thứ hai, giới hạn giá mà chúng ta nhận thấy trước đó hiển thị rõ ràng dưới dạng một đường nằm ngang ở mức 500,000. *But this plot reveals other less obvious straight lines*: a horizontal line around 450,000, một mức khác khoảng 350,000, *perhaps one around 280,000* và một số mức khác thấp hơn mức đó. Bạn có thể muốn thử xóa các quận tương ứng để ngăn thuật toán của bạn học cách tái tạo những đặc điểm dữ liệu này.

1.17 Thử nghiệm với sự kết hợp thuộc tính

Hy vọng rằng các phần trước đã cho bạn ý tưởng về một số cách để bạn có thể khám phá dữ liệu và hiểu rõ hơn. Bạn đã xác định được một số vấn đề dữ liệu mà bạn có thể muốn làm sạch trước khi cung cấp dữ liệu cho thuật toán Machine Learning và bạn đã tìm thấy mối tương quan thú vị giữa các thuộc tính, đặc biệt là với thuộc tính đích. Bạn cũng nhận thấy rằng một số thuộc tính có phân bố nặng về đuôi, vì vậy bạn có thể muốn biến đổi chúng (ví dụ: bằng cách tính logarit của chúng). Tất nhiên, quãng đường của bạn sẽ thay đổi đáng kể theo từng dự án, nhưng ý tưởng chung là tương tự nhau.

Điều cuối cùng bạn có thể muốn làm trước khi chuẩn bị dữ liệu cho thuật toán Machine Learning là thử các kết hợp thuộc tính khác nhau. Ví dụ, tổng số phòng trong một quận sẽ không hữu ích lắm nếu bạn không biết có bao nhiêu hộ gia đình. Điều bạn thực sự muốn là số phòng cho mỗi hộ gia đình. Tương tự, bản thân tổng số phòng ngủ không hữu ích lắm: bạn có thể muốn so sánh nó với số phòng. Và dân số mỗi hộ gia đình cũng có vẻ như là một sự kết hợp thuộc tính thú vị để xem xét. Hãy tạo các thuộc tính mới này:

Và bây giờ chúng ta hãy nhìn lại matrix tương quan:

Này, không tệ! Thuộc tính `bedrooms_per_room` mới có mối tương quan nhiều hơn với giá trị ngôi nhà trung bình so với tổng số phòng hoặc phòng ngủ. Rõ ràng những ngôi nhà có tỷ lệ phòng ngủ/phòng thấp hơn có xu hướng đắt hơn. Số phòng trong mỗi hộ gia đình cũng mang lại nhiều thông tin hơn tổng số phòng trong một quận – rõ ràng là nhà càng lớn thì giá càng đắt.

Vòng khám phá này không cần phải hoàn toàn thấu đáo; vấn đề là hãy bắt đầu một cách thuận lợi và nhanh chóng đạt được những hiểu biết sâu sắc giúp bạn có được nguyên mẫu đầu tiên khá tốt. Nhưng đây là một quá trình lặp đi lặp lại: sau khi thiết lập và chạy nguyên mẫu, bạn có thể phân tích kết quả đầu ra của nó để hiểu rõ hơn và quay lại bước khám phá này.

1.18 Chuẩn bị dữ liệu cho thuật toán Machine Learning

Đã đến lúc chuẩn bị dữ liệu cho thuật toán Machine Learning của bạn. Thay vì thực hiện việc này một cách thủ công, bạn nên viết các hàm cho mục đích này vì một số lý do chính đáng:

- Điều này sẽ cho phép bạn tái tạo các chuyển đổi này một cách dễ dàng trên bất kỳ dataset nào (ví dụ: lần tiếp theo bạn nhận được dataset mới).
- Bạn sẽ dần dần xây dựng một thư viện các hàm chuyển đổi mà bạn có thể sử dụng lại trong các dự án sau này.
- Bạn có thể sử dụng các chức năng này trong hệ thống trực tiếp của mình để chuyển đổi dữ liệu mới trước khi đưa dữ liệu đó vào thuật toán của bạn.
- Điều này sẽ giúp bạn có thể dễ dàng thử các phép biến đổi khác nhau và xem sự kết hợp các phép biến đổi nào hoạt động tốt nhất.

1.19 Làm sạch dữ liệu

Hầu hết các thuật toán Machine Learning không thể hoạt động khi thiếu features, vì vậy hãy tạo một vài hàm để xử lý chúng. Trước đó chúng ta đã thấy rằng thuộc tính `total_bedrooms` có một số giá trị bị thiếu, vì vậy hãy khắc phục điều này. Bạn có ba lựa chọn:

1. Loại bỏ các quận tương ứng.

2. Loại bỏ toàn bộ thuộc tính.

3. Đặt các giá trị thành một giá trị nào đó (không, giá trị trung bình, trung vị, v.v.).

Bạn có thể thực hiện những điều này một cách dễ dàng bằng cách sử dụng các phương thức `dropna()`, `drop()` và `fillna()` của `DataFrame`:

Nếu bạn chọn tùy chọn 3, bạn nên tính giá trị trung bình trên training set và sử dụng nó để điền các giá trị còn thiếu trong training set. Đừng quên lưu giá trị trung bình mà bạn đã tính toán. Sau này, bạn sẽ cần nó để thay thế các giá trị bị thiếu trong test set khi bạn muốn đánh giá hệ thống của mình và cả khi hệ thống hoạt động để thay thế các giá trị bị thiếu trong dữ liệu mới.

Scikit-Learn cung cấp một lớp tiện dụng để xử lý các giá trị còn thiếu: `SimpleImputer`. Đây là cách sử dụng nó. Trước tiên, bạn cần tạo một phiên bản `SimpleImputer`, chỉ định rằng bạn muốn thay thế các giá trị còn thiếu của từng thuộc tính bằng giá trị trung bình của thuộc tính đó:

Vì trung vị chỉ có thể được tính trên các thuộc tính số nên bạn cần tạo một bản sao của dữ liệu không có thuộc tính văn bản `ocean_proximity`:

Bây giờ bạn có thể khớp phiên bản máy tính với dữ liệu training bằng phương thức `fit()`:

Máy tính chỉ đơn giản tính giá trị trung bình của mỗi thuộc tính và lưu kết quả vào biến đối tượng `stats_` của nó. Chỉ thuộc tính `total_bedrooms` bị thiếu giá trị, nhưng chúng ta không thể chắc chắn rằng sẽ không có bất kỳ giá trị nào bị thiếu trong dữ liệu mới sau khi hệ thống đi vào hoạt động, vì vậy sẽ an toàn hơn khi áp dụng máy tính cho tất cả các thuộc tính số:

Bây giờ bạn có thể sử dụng máy tính “đã được training” này để biến đổi training set bằng cách thay thế các giá trị bị thiếu bằng các giá trị trung bình đã học:

Kết quả là một mảng NumPy đơn giản chứa features đã được chuyển đổi. Nếu bạn muốn đưa nó trở lại pandas `DataFrame` thì đơn giản:

1.20 Thiết kế Scikit-Learn

API của Scikit-Learn được thiết kế rất tốt. Đây là những nguyên tắc thiết kế chính:¹⁷

Tính nhất quán Tất cả các đối tượng đều có chung một giao diện nhất quán và đơn giản:

Transformer Một số estimators (chẳng hạn như máy tính) cũng có thể biến đổi dataset; chúng được gọi là máy biến áp. Một lần nữa, API rất đơn giản: việc chuyển đổi được thực hiện bằng phương thức `Transform()` với dataset để chuyển đổi thành một

Việc không phổ biến các lớp Datasets được biểu diễn dưới dạng mảng NumPy hoặc matrix thưa thớt SciPy, thay vì các lớp tự chế. Hyperparameters chỉ là các chuỗi hoặc số Python thông thường.

Thành phần Các khối xây dựng hiện có được tái sử dụng nhiều nhất có thể. Ví dụ: thật dễ dàng để tạo Pipeline estimator từ một chuỗi máy biến áp tùy ý, theo sau là estimator cuối cùng, như chúng ta sẽ thấy.

Mặc định hợp lý Scikit-Learn cung cấp các giá trị mặc định hợp lý cho hầu hết parameters, giúp dễ dàng nhanh chóng tạo hệ thống làm việc cơ bản.

1.21 Xử lý thuộc tính văn bản và classification

Cho đến nay chúng ta mới chỉ xử lý các thuộc tính số, nhưng bây giờ hãy xem xét các thuộc tính văn bản. Trong dataset này, chỉ có một: thuộc tính `ocean_proximity`. Hãy xem

giá trị của nó trong 10 trường hợp đầu tiên:

Đây không phải là văn bản tùy ý: có một số lượng giới hạn các giá trị có thể có, mỗi giá trị đại diện cho một danh mục. Vì vậy thuộc tính này là thuộc tính classification. Hầu hết các thuật toán Machine Learning đều thích làm việc với các con số, vì vậy hãy chuyển đổi các danh mục này từ văn bản sang số. Để làm điều này, chúng ta có thể sử dụng lớp `OrdinalEncoder` của `Scikit-Learn`:¹⁹

Bạn có thể lấy danh sách các danh mục bằng cách sử dụng biến thể hiện `Category_`. Đó là danh sách chứa mảng danh mục 1D cho mỗi thuộc tính classification (trong trường hợp này là danh sách chứa một mảng vì chỉ có một thuộc tính classification):

Một vấn đề với cách biểu diễn này là thuật toán ML sẽ cho rằng hai giá trị gần nhau giống nhau hơn hai giá trị ở xa. Điều này có thể ổn trong một số trường hợp (ví dụ: đối với các danh mục được sắp xếp như “xấu”, “trung bình”, “tốt” và “xuất sắc”), nhưng rõ ràng là không đúng với cột `ocean_proximity` (ví dụ: danh mục 0 và 4 rõ ràng giống với danh mục 0 và 1 hơn). Để khắc phục vấn đề này, một giải pháp phổ biến

Lưu ý rằng đầu ra là matrix thưa thớt `SciPy`, thay vì mảng `NumPy`. Điều này rất hữu ích khi bạn có thuộc tính classification với hàng nghìn danh mục. Sau khi code hóa `onehot`, chúng ta nhận được một matrix có hàng nghìn cột và matrix chứa đầy các số 0 ngoại trừ một số 1 trên mỗi hàng. Việc sử dụng nhiều bộ nhớ để lưu trữ các số 0 sẽ rất lãng phí, do đó, thay vào đó, một matrix thưa thớt chỉ lưu trữ vị trí của các phần tử khác 0. Bạn có thể sử dụng nó hầu như giống như một mảng 2D bình thường,²¹ nhưng nếu bạn thực sự muốn chuyển đổi nó thành một mảng `NumPy` (dày đặc), chỉ cần gọi phương thức `toarray()`:

Một lần nữa, bạn có thể lấy danh sách các danh mục bằng cách sử dụng biến thể hiện `Category_` của bộ code hóa:

Nếu một thuộc tính classification có số lượng lớn các danh mục có thể có (ví dụ: code quốc gia, nghề nghiệp, loài), thì code hóa một lần sẽ dẫn đến số lượng lớn features đầu vào. Điều này có thể làm chậm quá trình training và làm giảm hiệu suất. Nếu điều này xảy ra, bạn có thể muốn thay thế đầu vào classification bằng features số hữu ích liên quan đến các danh mục: ví dụ: bạn có thể thay thế `ocean_proximity` feature bằng khoảng cách tới đại dương (tương tự, code quốc gia có thể được thay thế bằng dân số của quốc gia đó và GDP bình quân đầu người). Ngoài ra, bạn có thể thay thế mỗi danh mục bằng một vector có chiều thấp, có thể học được gọi là nhúng. Cách trình bày của mỗi danh mục sẽ được học trong quá trình training. Đây là một ví dụ về học biểu diễn (xem Chương 13 và 17 để biết thêm chi tiết).

1.22 Custom Transformer

Ví dụ: đây là một lớp transformer nhỏ có thêm các thuộc tính kết hợp mà chúng ta đã thảo luận trước đó:

Trong ví dụ này, transformer có một hyperparameter, `add_bedrooms_per_room`, được đặt thành `True` theo mặc định (việc cung cấp các giá trị mặc định hợp lý thường rất hữu ích). Siêu tham số này sẽ cho phép bạn dễ dàng tìm hiểu xem việc thêm thuộc tính này có giúp ích cho thuật toán Machine Learning hay không. Tổng quát hơn, bạn có thể thêm hyperparameter để chuyển bất kỳ bước chuẩn bị dữ liệu nào mà bạn không chắc chắn 100%. Bạn càng tự động hóa các bước chuẩn bị dữ liệu này thì bạn càng có thể tự động thử nhiều kết hợp hơn, khiến bạn có nhiều khả năng tìm thấy một kết hợp tuyệt vời hơn (và giúp bạn tiết kiệm rất nhiều thời gian).

1.23 Chia tỷ lệ tính năng

Có hai cách phổ biến để có được tất cả các thuộc tính có cùng tỷ lệ: chia tỷ lệ tối thiểu-tối đa và tiêu chuẩn hóa.

Tiêu chuẩn hóa khác: đầu tiên nó trừ giá trị trung bình (vì vậy các giá trị tiêu chuẩn hóa luôn có trung bình bằng 0), và sau đó nó chia cho độ lệch chuẩn sao cho phân phối kết quả có phương sai đơn vị. Không giống như chia tỷ lệ tối thiểu-tối đa, tiêu chuẩn hóa không giới hạn giá trị trong một phạm vi cụ thể, điều này có thể là vấn đề cho một số thuật toán (ví dụ: các neural network thường mong đợi giá trị đầu vào nằm trong khoảng từ 0 đến 1). Tuy nhiên, tiêu chuẩn hóa ít bị ảnh hưởng bởi các giá trị ngoại biên hơn. Ví dụ: giả sử một quận có thu nhập trung bình bằng 100 (bởi lỗi). Chia tỷ lệ tối thiểu-tối đa sẽ nghiền nát tất cả các giá trị khác từ 0–15 xuống 0–0.15, trong khi tiêu chuẩn hóa sẽ không bị ảnh hưởng nhiều. Scikit-Learn cung cấp một bộ chuyển đổi có tên `StandardScaler` cho tiêu chuẩn hóa.

Như với tất cả các phép biến đổi, điều quan trọng là chỉ khớp bộ chia tỷ lệ với dữ liệu training chứ không phải với dataset đầy đủ (bao gồm cả test set). Chỉ khi đó bạn mới có thể sử dụng chúng để chuyển đổi training set và test set (và dữ liệu mới).

1.24 Đường ống chuyển đổi

Như bạn có thể thấy, có nhiều bước chuyển đổi dữ liệu cần được thực hiện theo đúng thứ tự. May mắn thay, Scikit-Learn cung cấp lớp `Pipeline` để trợ giúp các chuỗi chuyển đổi như vậy. Đây là một pipeline nhỏ cho các thuộc tính số:

Khi bạn gọi phương thức `fit()` của pipeline, nó gọi `fit_transform()` tuần tự trên tất cả các máy biến áp, chuyển đầu ra của mỗi cuộc gọi dưới dạng parameter sang cuộc gọi tiếp theo cho đến khi đạt đến estimator cuối cùng mà nó gọi phương thức `fit()`.

Pipeline hiển thị các phương pháp tương tự như estimator cuối cùng. Trong ví dụ này, estimator cuối cùng là `StandardScaler`, là transformer, do đó pipeline có phương thức `transform()` áp dụng tất cả các phép biến đổi cho dữ liệu theo trình tự (và tất nhiên cũng là phương thức `fit_transform()`, phương thức chúng ta đã sử dụng).

Cho đến nay, chúng ta đã xử lý các cột classification và cột số một cách riêng biệt. Sẽ thuận tiện hơn nếu có một transformer duy nhất có thể xử lý tất cả các cột, áp dụng các phép biến đổi thích hợp cho mỗi cột. Trong phiên bản 0.20, Scikit-Learn đã giới thiệu `ColumnTransformer` cho mục đích này và tin tốt là nó hoạt động rất tốt với pandas `DataFrames`. Hãy sử dụng nó để áp dụng tất cả các phép biến đổi cho dữ liệu nhà ở: